SCE WORKING GROUP 1 · PROJECT A · OCTOBER 2026

# Semantic ontologies

Formal descriptions

Akshay Shanker

# Understanding AI

**Premise.** AI can now write code, translate papers to code and modify our models (Wu et al. 2022).

**What remains for us.** Two questions remain:

- Do we and AI agree on the meaning of a paper or a model description?
- Can we inspect and verify what AI understands?

## Semantics

Both are questions about meaning, or **semantics**.

- Meaning is a relation between a piece of text and some thing outside language.
- A system trained on text alone cannot contain the relationship of the text to meaning (Bender and Koller 2020).
- A large language model appeats to approximate the map from text to meaning statistically rather than structurally.

# Semantic ontology

We can write meaning down in a form a machine reads, as a semantic ontology.

- A semantic ontology is a machine-readable account of objects, their types and the valid relations between them.
- Structural model of the model.
- AI can be trained on a semantic ontology, or can answer from the ontology's program outputs, as when it writes a model in Lean and answers from the kernel's output. Other ways AI can work with ontologies are listed in the appendix.

# The problem

The problem

# Example 1: Buffer-stock saving

# Is a semantic ontology just the mathematics of the model?

Consider a written statement of the buffer-stock saving model.

> *Each period the household arrives with assets $k_t$, receives income $y_t$ and earns the gross return $R\,k_t$ on those assets, where $R$ is the gross interest factor. Market resources are then $m_t = R\,k_t + y_t$. Income is i.i.d. with $\log y_t \sim N(\mu_y, \sigma_y^2)$. The household chooses consumption $c_t \in (0, m_t]$, leaving $k_{t+1} = m_t - c_t$, to solve the Bellman equation*
>
> $$v_t(m_t) = \max_{0<c_t\leq m_t} \left\{u(c_t) + \beta\,\mathbb{E}_t\, v_{t+1}\big(R\,(m_t - c_t) + y_{t+1}\big)\right\}.$$
>
> *The right-hand side defines an operator $\mathbb{T}$ with $v_t = \mathbb{T}\,v_{t+1}$, which sends the function $v_{t+1}$ to the function $v_t$, backward from a given terminal value over a finite horizon.*

- By itself, the statement does not give a machine enough information to build $\mathbb{T}$.
- An economist can nonetheless write code for $\mathbb{T}$ from the statement alone, by using their implicit understanding of Dynamic Programming.
- A semantic ontology makes that implicit understanding explicit.

# Missing Meaning Problem

Parsing the text describing the Buffer stock saving model misses semantics. See Appendix for more detail on parse trees.

| Missing meaning | In the buffer-stock statement |
|---|---|
| Which variables are bound, and where | The maximization binds $c_t$, while the expectation binds $y_{t+1}$. The left-hand side binds $m_t$ by defining $v_t$ as a function of $m_t$, which leaves $m_t$ free inside the right-hand side. |
| Parameters | The paper defines $u$, $R$, $\beta$ and the law of $y_{t+1}$ elsewhere, in no fixed place. Only $\beta$ and $R$ are numbers, $u$ is a function and the law is a measure - this is not immediately apparent from the statement. |
| Symbol types | The value function $v_t$ maps $\mathbb{R}_{++}$ to $\mathbb{R}$, and $m_t$ a point of in its domain. The tree records both symbols but not the fact that $v_t(m_t)$ is a function evaluation at a variable of a particular type. |
| An equation between functions | For each $m_t$ the Bellman equation equates two numbers. Over all $m_t$ it is one equation between functions, $v_t = \mathbb{T}\, v_{t+1}$, in which $\mathbb{T}$ maps $v_{t+1}$ to $v_t$. |

# Missing Meaning Problem

| Missing meaning | In the buffer-stock statement |
| --- | --- |
| Whether a symbol is a variable, a function, or a random variable | The symbol $m_t$, defined by $m_t = R\,k_t + y_t$, is i) the path of one household's resources, ii) the function $(k, y) \mapsto Rk + y$ composed inside the operator, and iii) the random variable whose law is the cross-sectional distribution. A paper writes one symbol for all three meanings, whereas code must represent them as three objects of different types "doing" different things. |
| Shock law | The statement fixes $\log y_t \sim N(\mu_y, \sigma_y^2)$. In the shorter form $y \sim \text{LogNormal}(\mu_y, \sigma_y)$, $\mu_y$ denotes the mean of $\log y$ under one convention and the mean of $y$ under another. |

# Syntax omits meaning: timing and information

| Missing meaning | In the buffer-stock statement |
| --- | --- |
| Implications of timing stated in words | "A policy shock raises entrants at date $t$." If the entrants change $y_t$ in equilibrium, the sentence is consistent with a model in which prices respond at $t$ and with one in which they respond only at $t+1$. |
| What the household knows when it decides | If the return $R_t$ is random, persistent and drawn before the decision, the household's expectation of $R_{t+1}$ depends on whether it observes $R_t$ or only $m_t = R_t\, k_t + y_t$ when it decides. |

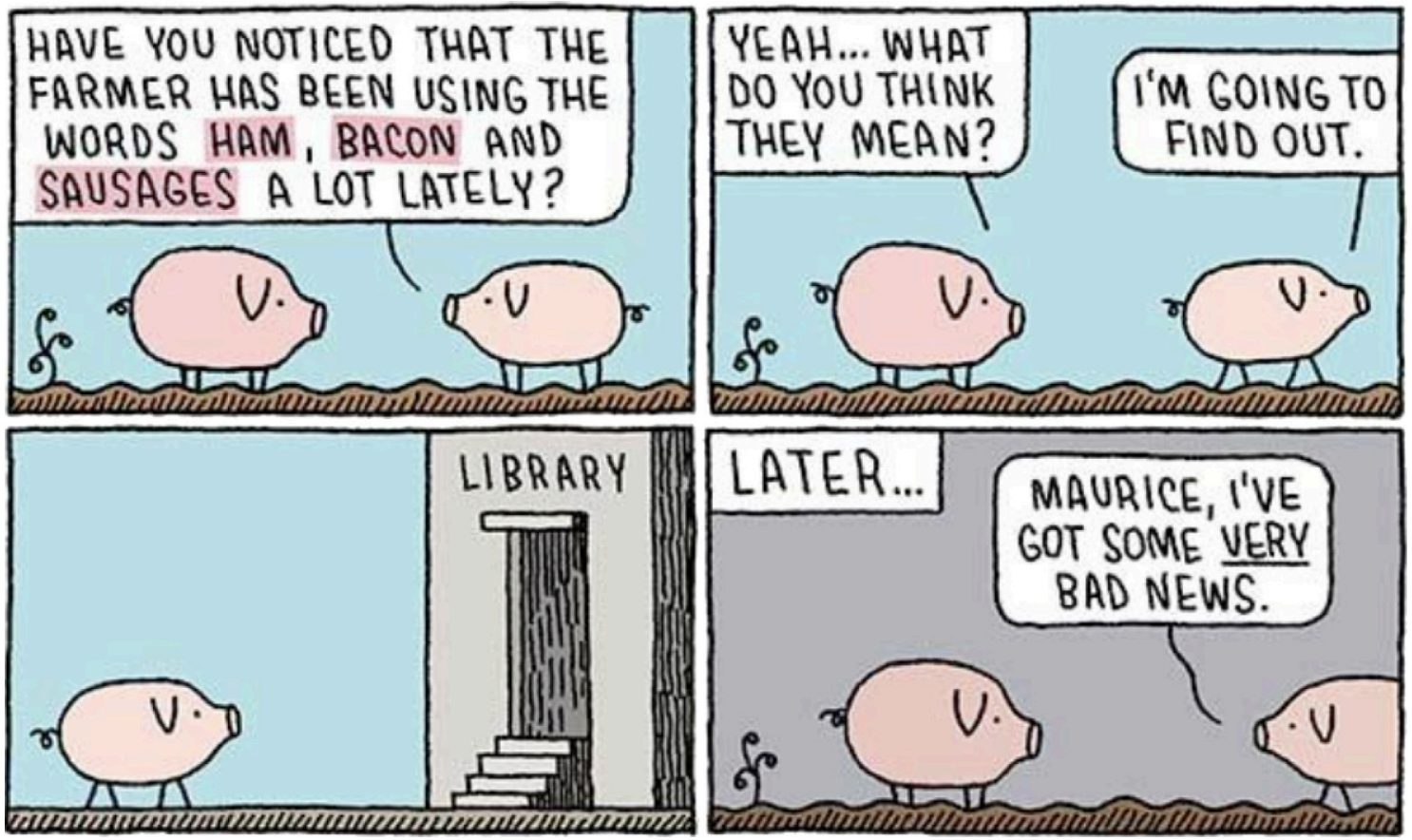
## Multiple possible parsed meanings

- One well-formed statement could admit two or more different readings of the parse tree. Appendix slides gives examples of false agreement.
- False agreement does not usually arise when a human interprets the model. E.g., an economist has been taught to know that $m_t$ when stated in $v(m_t)$ is defining the evaluation of $v$, and is not an asset path across a population of market resources.
- Although...interpreting the timing of events, and what the individual knows at each decision, nevertheless confuse economists themselves.

The problem is when we rely on AI's statistical interpretation of text, we can never be sure that AI understands the model correctly; even if it appears to do so "most" of the time.

# Example 2: the pig in the library

# What the pig learned in the library




© Tom Gauld, reproduced in Guizzardi and Guarino (2024), "Explanation, semantics, and ontology", *Data & Knowledge Engineering* 153, 102325, Figure 1; shown here for discussion within the working group.

# The semantics is not enough

What would it take to understand the joke? Semantics is not enough. Semantics is a function from symbols to a domain, whereas the ontology is the system of kinds and relations that the symbols are about (Guizzardi and Guarino 2024, §2.1). The joke here arises from the system, not the semantics.

| | **The pigs** | **Buffer stock model** |
|---|---|---|
| The written form | $ham,\ bacon,\ sausages$ | $m,\ y,\ \mathbb{E},\ v,\ \mathbb{T}$ in the paper's Bellman equation |
| The formal semantics, the function $[\![\cdot]\!]$ from symbols to the domain | $[\![ham]\!],\ [\![bacon]\!],\ [\![sausages]\!] \in \mathsf{Food}$, the dictionary look-up | $[\![\text{Bellman equation}]\!] = \mathbb{T}$, some operator, with the space it acts on, the scope of each binder and the law of $y$ unstated |
| What that semantics leaves open | whether the three words name foods for pigs or foods from pigs | which variable each of $\max$ and $\mathbb{E}$ binds, whether $v$ is a function and on which space, the law of $y$, when $y$ is drawn, and what the household knows when it chooses |
| The ontology, the kinds and relations, written as typed symbols, each with a signature, what it takes and what it returns | $Maurice : \mathsf{Pig}$<br>$madeOf : \mathsf{Ham} \to \mathsf{Pork}$<br>$from : \mathsf{Pork} \to \mathsf{Pig}$ | $v : \mathbb{R}_{++} \to \mathbb{R},\ y : \mathbb{R}_{++}$ with law $\nu_y$<br>$\mathbb{T} : (\mathbb{R}_{++} \to \mathbb{R}) \to (\mathbb{R}_{++} \to \mathbb{R})$<br>$\mathbb{T} = \max_c \circ \mathbb{E}_y$, the expectation taken first |
| False agreement | Two pigs who agree that the words name foods, one meaning foods for pigs and the other foods from pigs, and never go to the library | Two readers who accept the Bellman equation but disagree on timing of price response to shocks. |

## The answer

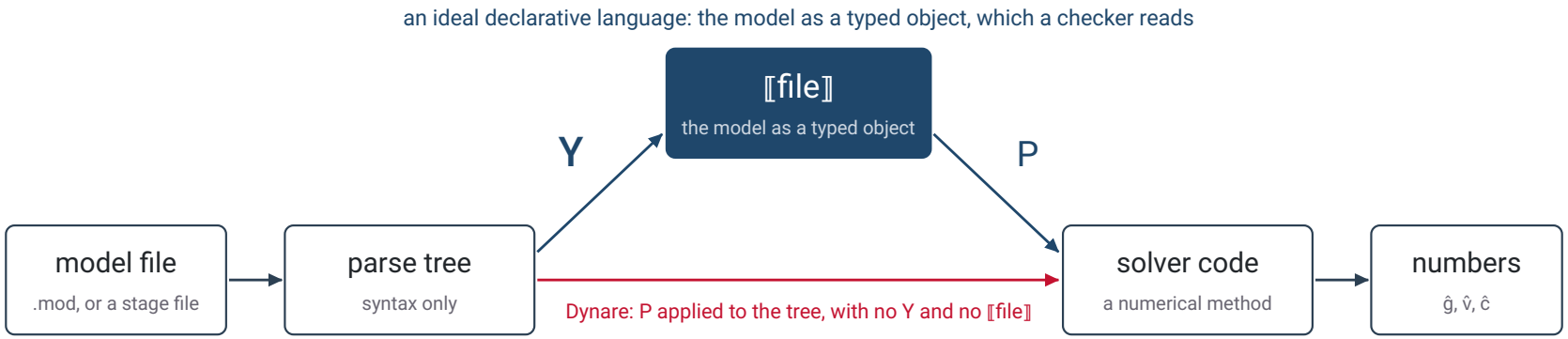
A semantic ontology is the mathematics of the model, with the types, binders and laws that the paper's Bellman equation leaves to the reader, stated in a machine-readable file.

- An economist supplies the missing meaning from implicit context, the conventions of a subfield and the paper's surrounding prose (slides 8 to 10).
- An AI may *seem* to supply the same meaning, although it draws on the statistics of its training data rather than on a deterministic set of rules.
  - The reading it returns is stochastic, and producing it is expensive, since the AI must search the paper and similar papers for context.

# What does DYNARE do?

## Without semantic ontologies, how does DYNARE work?

Both routes parse the file into a tree that holds syntax only, and both end in solver code. An ideal declarative language first builds ⟦file⟧, the model as a typed object, by Y (upsilon), and then attaches a numerical method to it by P (rho). Dynare applies P to the tree directly, so its Y is never written and the interpretation is attached implicitly, inside the numerical implementation.




**The meaning of a `.mod` file.** It is what the program does with the file: "The Dynare Preprocessor defines the Dynare model language" (the preprocessor's README).

# What an AI needs beyond Dynare

**Dynare does what it was built for.** One program, one solver and one set of conventions, learned once, have given economists the right numbers from a short file since Juillard (1996).

**Its meaning is tied to its numerics.** The model has no statement apart from the perturbation solver, so a model moved to HARK or to the sequence-space Jacobian toolkit can be compared with it only through the numbers.

**An AI writing new code needs more.** Its file is checked for syntax, for one equation per variable and for a steady state, not for what it means. New code, for a new model or another toolkit, needs the model stated apart from any solver, where a checker can read and refuse it.

# Examples of semantic ontologies

# Three parts to a semantic ontology

$$\text{written form} \xrightarrow{\Upsilon} \text{domain of meaning}$$

**The written form.** Machine-readable symbols, each with a type and a signature, what it takes and what it returns, as in $v : X \to \mathbb{R}$.

**The domain of meaning.** The objects we model, sorted into kinds with relations between them: state spaces, laws, value functions and operators, as the kinds of slide 14 are Pig, Ham and Pork.

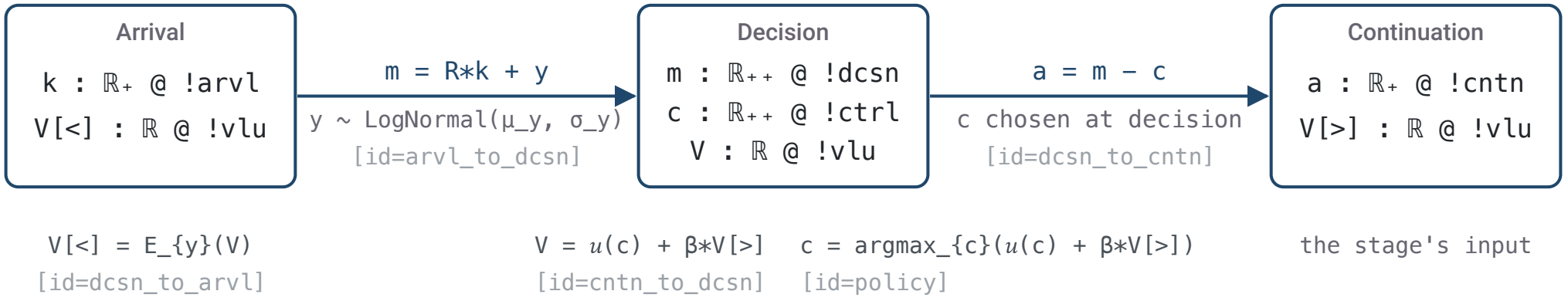
**The map Y (upsilon).** It sends a symbol of a given type to an object of the matching kind, and a program checks that it does. Economists already have the domain, since their models are well described. Project A supplies the written form and the map.

Harel and Rumpe (2004), "Meaningful modeling: what's the semantics of 'semantics'?", *Computer* 37(10), 64–72, for the three parts · Bellman calculus, Introduction §1 (bellman-ddsl/docs) · in this deck a type belongs to the written form and a kind to the domain; Guizzardi and Guarino (2024) write "type" for what the deck calls a kind.

# Economics: one stage of the buffer-stock problem as a stage file

Project Bellman writes one period of the problem as a stage file in the language Bellman-SYM.

**Written form, the typed stage file**




**Perches.** The three information sets of the period: arrival `!arvl`, decision `!dcsn` and continuation `!cntn`.

**Fields.** A state or a value declared at a perch with its set, as in `m : ℝ₊₊ @ !dcsn`.

**Blocks.** The equations between two perches: the transition with the shock, the choice, and the expectation.

Bellman-SYM, Stages §§1–4 (the documentation example, in the letters k, m, a of the calculus); Bellman calculus, Stages and perches §1 (bellman-ddsl/docs) · the law line fixes the convention that slide 9 left to a subfield.

# Economics: what the stage file refuses or makes visible

Y (upsilon) reads each field and block of the file as a typed object. The elaborator, the program that constructs those objects, refuses a line that the types do not admit and names the rule.

| The file writes | Under Y it denotes |
|---|---|
| `V[>] : ℝ @ !vlu` at the continuation perch | $v_{\succ} : \mathbb{R}_{+} \to \mathbb{R}$, a function of savings, never a number |
| the choice block `[id=cntn_to_dcsn]` with `policy` | $\mathbb{T}_{\succ} : \mathsf{V}_{\succ} \to \mathsf{V}$, the choice operator |
| the expectation block `[id=dcsn_to_arvl]` | $\mathbb{T}_{\prec} : \mathsf{V} \to \mathsf{V}_{\prec}$, the expectation operator |

| A line that | The elaborator |
|---|---|
| reads the shock beside its expectation, `V[<] = E_{y}(V) + y` | refuses it under `shock-reintroduced`, since $y$ is not known at arrival |
| lists a continuation value at the decision perch | refuses it under `field-perch`, since the value is a function of savings |
| puts the mean of the shock in the transition, `m = R*k + exp(μ_y + σ_y^2/2)` | accepts it, and the different model stands visible on that line |
| writes a second `β` in the expectation block | accepts it, and the second `β` stands visible on its own line |
| would impose a stationary closure $v_{t+1} = v_t$ | has no line for it, since `V[>]` comes from the next period or the terminal boundary |

The appendix, slides 40 to 50, gives the full file, each refusal with its rule, and the period and trellis files.

The reference on refusals, rules shock-reintroduced and field-perch; Bellman-SYM, Field equations §5, Perch tagging rules §3, Periods §§2–3, Trellises §2; the runs of `elaborate_text` of 13 October 2026 listed on the appendix slides.

# Biology: the Gene Ontology

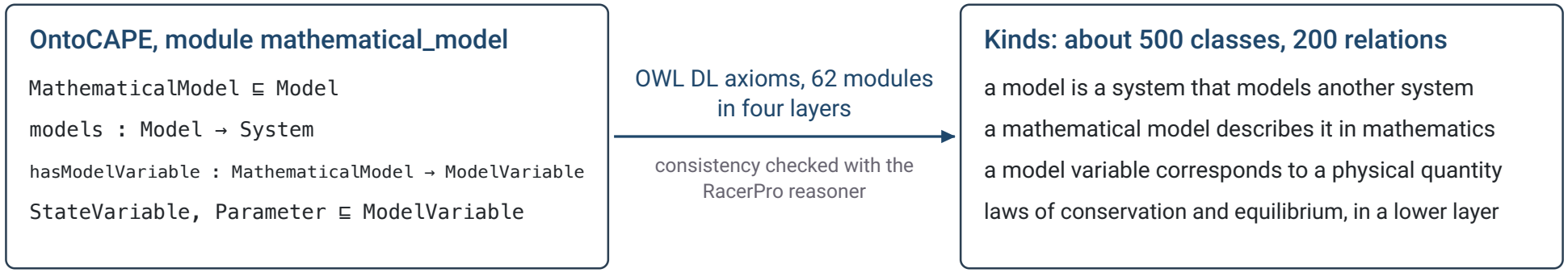



**Excludes.** A missing annotation read as the absence of a function (the open-world assumption), a NOT read as an inconclusive result, and a term read by its name rather than by its definition and relations, since a label must be unique and a definition must always denote the same referents (OBO Foundry principles 12 and 19).

**For economics.** A model's object is named by a term with a definition and relations, not by a variable name. The role tags `!ctrl` and `!vlu` and the law line of a stage file play that part, as the `sboTerm` of the Systems Biology Ontology does for a rate law in SBML.

Ashburner et al. (2000), "Gene Ontology: tool for the unification of biology", *Nature Genetics* 25(1), 25–29 · The Gene Ontology Consortium (2026), "The Gene Ontology knowledgebase in 2026", *Nucleic Acids Research* 54(D1), D1779–D1792, Table 3 (the release of 22 July 2025) · GAF 2.2 documentation and the human annotation file of 21 May 2026 (geneontology.org); the go-ontology build file, which runs the ELK reasoner · SBO: Courtot et al. (2011), *Molecular Systems Biology* 7, 543.

# Engineering: OntoCAPE, the ontology for chemical process engineering




**Excludes.** A model confused with the system it models, and a model variable confused with the physical quantity it stands for: the relation `models` joins a model to its system, and `correspondsToQuantity` pairs a variable with a quantity, as a declared domain excludes the deck's function on the wrong space.

**For economics.** OntoCAPE is the nearest case to Project A, an ontology of models themselves: model variables, equation systems and solution strategies are kinds in its modules, so that a process model and its solver are distinct typed objects, as Y and P keep a model apart from its numerical method.

Morbach, Yang and Marquardt (2007), "OntoCAPE—A large-scale ontology for chemical process engineering", *Engineering Applications of Artificial Intelligence* 20(2), 147–161 · Morbach, Wiesner and Marquardt (2009), *Computers & Chemical Engineering* 33(10), 1546–1556 · Marquardt, Morbach, Wiesner and Yang (2010), *OntoCAPE: A Re-Usable Ontology for Chemical Process Engineering*, Springer · the OntoCAPE 2.0 distribution (AVT, RWTH Aachen, GPL): system.owl and mathematical_model.owl; technical reports LPT-2008-25 and LPT-2008-28; Morbach (2009), dissertation, RWTH Aachen, on the RacerPro checks.

# Earth science: the SWEET ontologies




**Excludes.** A quantity name read as any of several kinds: a SWEET class fixes the kind by its definition and by its place among realms, quantities and phenomena. The CF standard names, published as SKOS concepts with their canonical units on the NERC vocabulary server, settle the same question for a netCDF variable such as `temp`, which is the law behind a parameter and timing stated in words in the deck's terms.

**For economics.** The comparison paper of the special issue that Project A proposes needs a vocabulary of this kind: which object each team reports, a value or a marginal value function, on which state and under which timing (den Haan, Judd and Juillard 2010, 2011).

Raskin and Pan (2005), "Knowledge representation in the semantic web for Earth and environmental terminology (SWEET)", *Computers & Geosciences* 31(9), 1119–1125 · ESIPFed/sweet, release 3.5.0 (14 July 2022), the header of sweetAll.ttl; the class count from the 3.5.0 source · CF standard names: NERC Vocabulary Server, collection P07, version 95 (16 September 2026), publisher Natural Environment Research Council, owner the CF Standard Names Committee · den Haan, Judd and Juillard (2010, 2011), *Journal of Economic Dynamics and Control* 34(1), 1–3 and 35(2), 175–177.

# Finance: FIBO, the Financial Industry Business Ontology

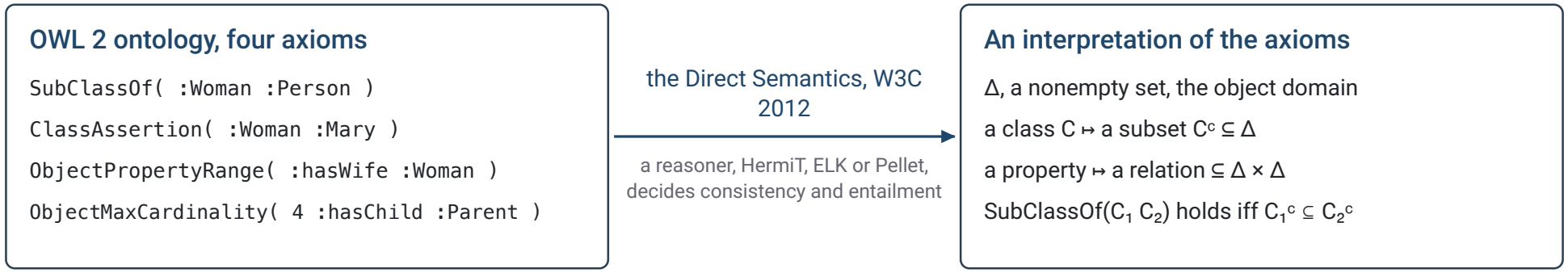



**Excludes.** An issuer read as the obligor of a bond, and a notional amount read as a principal: each is a defined class or property, and the 2023 revision of the Swaps ontology separated the nominal from the notional amount, which its change note says "were confused". In the deck's terms, a function on the wrong space and the law behind a parameter.

**For economics.** FIBO is finance's own ontology, with the Bureau of Economic Analysis, the Bureau of Labor Statistics, the CFTC and the Office of Financial Research among its contributors, and the 2026 rule under the Financial Data Transparency Act records it among the standards that commenters proposed. Project A's ontology names the objects of a model as FIBO names the parties and terms of a contract.

EDM Council, FIBO (spec.edmcouncil.org/fibo), release Q2 2026 (14 July 2026), MIT licence; the development-process page on consistency; onto-viewer's consistency checker (Openllet and HermiT) · OMG specifications EDMC-FIBO/FND 1.2, BE 1.1, FBC 1.0 and IND 1.0 (2017–2018) · Bennett (2013), "The financial industry business ontology: best practice for big data", *Journal of Banking Regulation* 14(3–4), 255–268 · Allemang, Garbacz, Grądzki, Kendall and Trypuz (2021), *FOIS 2021*, 112–126 · Financial Data Transparency Act Joint Data Standards, 91 FR 38246 (25 June 2026) · GoodRelations, the ontology schema.org absorbed in 2012: Hepp (2008), *EKAW 2008*, LNCS 5268, 329–346.

# Knowledge representation: OWL 2, the language ontologies are written in




**Excludes.** Any reading of the axioms other than the one the Direct Semantics fixes: "the correct answer to any such question is predetermined by the formal semantics", whatever reasoner computes it (Primer §2). Nor is a missing statement read as false, or two names as two individuals.

**For economics.** This is the three-part definition in full: classes, properties and individuals as typed symbols, an interpretation as the domain, and a map whose consistency and entailments a reasoner decides. A stage file's Y plays the part of the Direct Semantics, and the elaborator the part of the reasoner.

W3C (2012), *OWL 2 Web Ontology Language*, Recommendations of 11 December 2012: Document Overview §2.3; Direct Semantics §§2.2–2.5; Profiles §§1 and 5; Primer §§2, 4 and 9 · Baader, Calvanese, McGuinness, Nardi and Patel-Schneider (eds.) (2007), *The Description Logic Handbook*, 2nd ed., Cambridge University Press · HermiT: Glimm, Horrocks, Motik, Stoilos and Wang (2014), *Journal of Automated Reasoning* 53(3), 245–269; ELK: Kazakov, Krötzsch and Simančík (2014), *Journal of Automated Reasoning* 53(1), 1–61.

# The five cases against the three parts of a semantic ontology

Each row places one case against the three parts of slide 19, the typed symbols of the written form, the domain of meaning and the map between them, and names in the last column the program that checks the map.

| Case | Typed symbols | Domain | Map | Checker |
|---|---|---|---|---|
| Gene Ontology | the GO terms, each by ID | kinds of molecular function, biological process and cellular component | the term in a GAF annotation | the ELK reasoner on the ontology at every commit |
| OntoCAPE | OWL classes and relations in 62 modules | models, systems, materials and quantities | the axioms, `models` and `hasModelVariable` | the RacerPro reasoner |
| SWEET and the CF names | OWL classes, and SKOS concepts for the CF names | Earth-science quantities, realms and phenomena | the class, or the standard name on a variable | any OWL tool; the CF checker against the NVS table |
| FIBO | OWL classes and properties | contracts, instruments and parties | the class asserted of a record | two OWL 2 reasoners on every release |
| OWL 2 | classes, properties and individuals | an interpretation: a set with relations on it | the Direct Semantics | a reasoner: consistency and entailment |

Harel and Rumpe (2004), *Computer* 37(10), 64–72, for the three parts · the sources of each case are on its slide: Ashburner et al. (2000) and the Gene Ontology Consortium (2026); Morbach, Yang and Marquardt (2007); Raskin and Pan (2005) and the NERC Vocabulary Server; the EDM Council and Bennett (2013); W3C (2012).

# The stage file against the same three parts

The stage file of slide 42, the deck's own written form, is read here against the columns of the table on slide 27.

| Part | In the Bellman-SYM stage file |
|---|---|
| Typed symbols | the fields and the blocks, each with a type and a perch, and the declared law of the shock (slide 42) |
| Domain | state spaces, laws, value functions and the operators between them (slide 19) |
| Map | the perch tags `@ !arvl`, `@ !dcsn` and `@ !cntn`, the role tags `!ctrl` and `!vlu` and the law line (slide 42), interpreted by Y (upsilon), the checked interpretation of slide 36, as the objects of slide 19 |
| Checker | the elaborator (slide 45), which constructs the typed object or refuses the file, naming the rule the file breaks and the line; and the comparator, a program that compares two elaborated files up to renaming and reports, for each kind of item, agreement, difference or an undecided residue, the items it can neither match nor reject (slide 31) |
| Run on the file of slide 42 | The elaborator constructed the object from the file and refused the line `E_{y}(V) + y`, with `y` listed among the block's inputs, under `shock-reintroduced`.<br>The comparator read the file and the documentation example, with b and w for k and m, as one object under that renaming. |

# Kinds, individuals and the community library

# Kind and individual

$$\text{written form} \xrightarrow{\ \Upsilon\ } \underbrace{\mathbb{T}_{\prec} \circ \mathbb{T}_{\succ} : \mathsf{V}_{\succ} \to \mathsf{V}_{\prec}}_{\text{the kind: the model as a mathematical object}} \xrightarrow{\ \mathrm{P}\ } \underbrace{(\hat{\mathbb{T}}_{\prec} \circ \hat{\mathbb{T}}_{\succ})_{\mathrm{HARK}},\ (\hat{\mathbb{T}}_{\prec} \circ \hat{\mathbb{T}}_{\succ})_{\mathrm{SSJ}}}_{\text{individuals: the operator with grids, quadrature and interpolation attached}}$$

**In the cartoon's terms.** The taxonomy's entry Pig is a kind, of which Maurice is an individual, although the taxonomy itself contains no pig.

**The paper's model as kind and an implementation as individual.** P (rho) attaches grids, quadrature and interpolation to the operators that the kind names, as the display shows for HARK and for the sequence-space Jacobian toolkit, SSJ. A grid, the finite list of points at which the computer evaluates a function, is not the declared set and belongs with the method and settings files. A change of interpolation likewise changes the approximation and not the operation.

# One kind, several individuals

**Two files, one object.** Two files that differ in spelling denote one object under Y (upsilon), so that the comparator compares them as objects up to renaming, whereas two implementations of one object differ in approximation.

**What the comparator does not read.** The settings of P (rho), the grid, the quadrature and the interpolation, are declared in the method and settings files beside the stage file. The comparator's profile, the list of the kinds of item it compares, names the fields, parameters, functions, laws, blocks and stochastic kernels, the conditional distributions of the transitions, and none of those settings. Two implementations that differ only in grid, quadrature or interpolation are therefore one object to the comparator, with the difference visible only in their numbers.

**The question of slide 2.** When our reading of the paper is the stage file we write from it and the AI's reading is the file it writes, the comparator reports the first item on which the two files differ.

**A different law of the shock.** Two files that differ in the law of $y$ denote, under Y, two objects that differ in $\nu_y$ and agree elsewhere. The comparator's report then lists, for each kind of item, agreement, difference or an undecided residue, as slide 28 says.

# What ties an individual to its kind

**What the comparator reads.** The comparator reads two declarations, that is, two stage files. The code an AI writes from a declaration is tested only by the comparison of numbers on the next slide, because no program checks an implementation against its stage file.

**In the five cases the individual's record names its kind.** A GO annotation names a GO term, an OntoCAPE or FIBO record asserts its class, a variable carries a SWEET class or a CF standard name, and an OWL individual is asserted into a class, so that in each case a reasoner or a checker reads the kind from the record itself.

**In the community library the name is a link.** In the community library of Project B, the Community Library of Baseline Models (slide 33) an implementation names its model by a link to the shared model description and an optional metadata entry, `model: "BST"` , the library's short name for the buffer-stock model, and only the numbers of slide 33 test the implementation.

Community Library, buffer-stock submission guide, the optional metadata file · the cases: Ashburner et al. (2000), Morbach, Yang and Marquardt (2007), Raskin and Pan (2005), the EDM Council's FIBO and W3C (2012), on their slides · bellman-ddsl wiki, judge §4 (the comparator compares elaborated declarations).

# The community library holds the individuals

**Project B's library.** The library, which QuantEcon hosts, holds one model statement and one calibration per baseline model, implementations across toolkits, and results that readers can compare.

**The precedent, BioModels.** BioModels holds about 2,000 models in SBML. For the 800 curated models, the BioModels curators have checked the computational representation against the biology, checked that simulations reproduce the reference publication's results, and attached ontology terms (Malik-Sheriff et al. 2020).

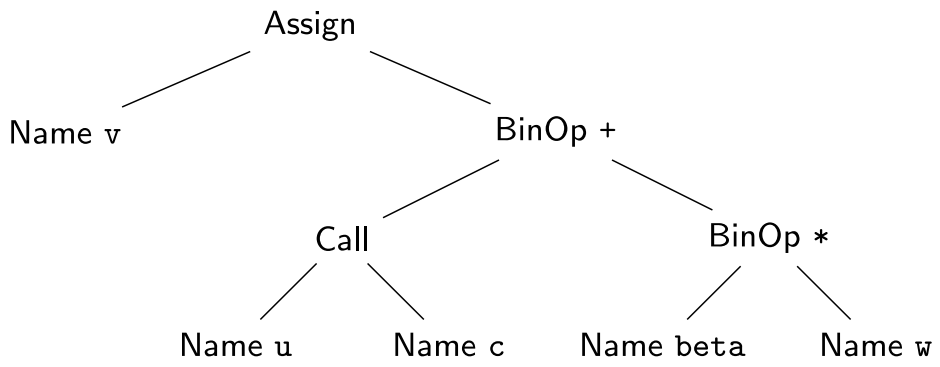
**What a comparison of numbers tests.** Because parsing, interpretation, method assignment, calibration and execution must all agree for a result to approximate the declared problem, agreement between two implementations is evidence about P (rho). Reproducing a published number, by contrast, tests one implementation. A misreading of the model is caught by the comparator of slide 31, whereas an error in the code is caught only here, by the numbers.

**What makes the test well posed.** The stated kind says which object both implementations claim to approximate. Without it, a library can be compared only number by number, a comparison in which the agreement of two numbers does not exclude the false agreement of slide 14.

# Appendix

# The parse tree of a written statement

Parsing a written statement into a tree, as every compiler first does, shows what the statement leaves to context.




**What the parser does.** It splits `v = u(c) + beta * w` into a tree by grammar rules alone.

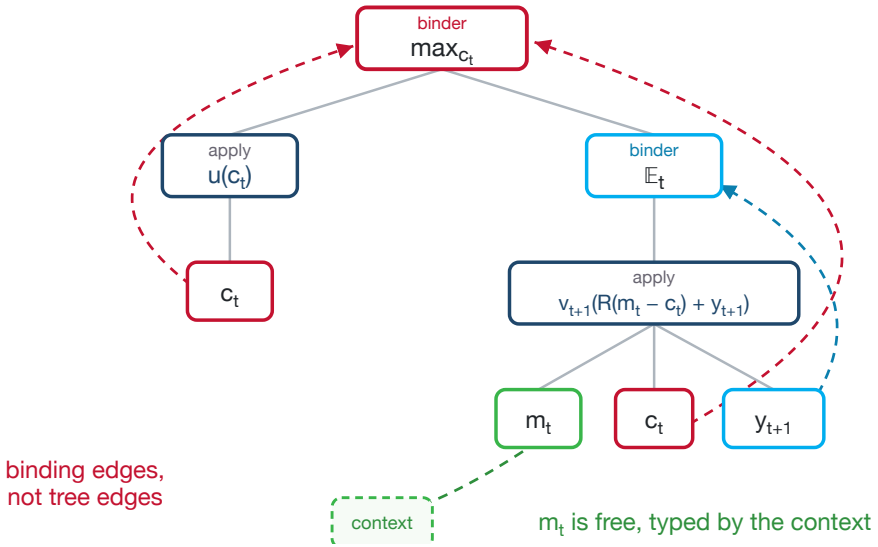
**What the tree records.** The tree records only which symbol is applied to which, node by node.

**What the tree omits.** The tree records nothing about what `v`, `c` or `w` are, which variable is bound where, or whether `v` is a function.

The tree is Python's abstract syntax tree, `ast.parse("v = u(c) + beta * w")`.

# From the syntax tree to the semantic graph

In the model, $v$ is not a number obtained by adding $\beta w$ to $u(c)$. That $v_t = \mathbb{T}\, v_{t+1}$ sends a function to a function appears only once the binders, types and declarations on the right are attached to the tree, the dotted lines in the figure.




**binders:** $\max_{c_t}$ binds $c_t$, $\quad \mathbb{E}_t$ binds $y_{t+1}$,
$v_t(m_t) = \cdots \quad \forall m_t \in \mathbb{R}_{++}$ binds $m_t$

**fixed from outside:** $u : \mathbb{R}_{++} \to \mathbb{R}, \quad \beta,\, R \in \mathbb{R},$
$\nu_y$ the law of $y_{t+1}$, on $\mathbb{R}_{++}$

**obtained:** $v_{t+1}\big(R(m_t - c_t) + y_{t+1}\big) : \mathbb{R},$
$\mathbb{T} : (\mathbb{R}_{++} \to \mathbb{R}) \to (\mathbb{R}_{++} \to \mathbb{R})$

The law $\nu_y$ is the probability distribution of the income shock. A semantic ontology states the quantifier over $m_t$ on the left-hand side, the binder conventions, the types and the declarations of $u$ and $\nu_y$, none of which the tree records.

Bellman-SYM, Project Bellman's model language, parses a file into such a tree, builds the semantic graph from it by a checked interpretation, which the calculus calls Y (upsilon), and discards the tree (Bellman calculus, Introduction §2; Bellman-SYM, Stages §5).

# What the expectation integrates

The paper's Bellman equation gives $\mathbb{E}$ the date $t$ as its subscript rather than the variable it integrates. Because the tree specifies neither that variable nor the type of $\mathbb{E}_t$, each box below puts the paper's expression to the left of "read as" and a reading that the tree does not exclude to the right.

### The shock leaves the expectation.

$$\mathbb{E}_t\, v\big(R(m-c)+y'\big) \qquad \text{read as} \qquad v\big(R(m-c)+\mathbb{E}_t\, y'\big) \quad \text{or} \quad \mathbb{E}_t\big[v\big(R(m-c)\big)\big]+y'.$$

Since Jensen's inequality gives $\mathbb{E}_t\, v(R(m-c)+y') \leq v(R(m-c)+\mathbb{E}_t\, y')$ for concave $v$, the first reading overstates the continuation value. Because it replaces the income risk by its mean, it also removes the precautionary motive, the saving held against that risk (Kimball 1990). The second reading treats $y'$ as known before it is drawn.

### The expectation moved through the maximization.

$$\max_c \big\{u(c)+\beta\, \mathbb{E}_t\, v_{t+1}(\cdot)\big\} \qquad \text{read as} \qquad \mathbb{E}_t \max_c \big\{u(c)+\beta\, v_{t+1}(\cdot)\big\}.$$

Since $\mathbb{E}\max \geq \max \mathbb{E}$, the reading on the right overstates the value and lets consumption respond to income that has not yet been drawn.



Jensen's inequality and the precautionary motive: Kimball (1990), "Precautionary saving in the small and in the large", *Econometrica* 58(1), 53–73.

# A number for a function, and a function on the wrong space

### A number where a function belongs.

$v_t = \mathbb{T}\, v_{t+1}, \quad v_t : \mathbb{R}_{++} \to \mathbb{R}$ read as $v_t(\hat{m}_t)$ alone, at $\hat{m}_t = R\,\hat{k}_t + \hat{y}_t$, one simulated household's resources.

The backward step then returns one number and one consumption $c_t(\hat{m}_t)$, although $\mathbb{T}$ needs $v_{t+1}$ at every $R(\hat{m}_t - c) + y'$. Under this reading the reader has simulated a path where the model asked for a function.

### A function on the wrong space.

$\mathbb{E}_t\, v_{t+1}(Ra + y')$ on savings $a = m - c$, the paper's $k_{t+1}$, $a \in [0, \infty)$ read as $v_{t+1}$ itself, on resources $m \in (0, \infty)$.

The borrowing limit acts at $a = 0$, a point at which a function of resources is not defined, so that the two functions have different domains. A reading that confuses them puts the boundary at a point where the constraint does not act.

# The discount and the expectation applied twice, and the recursion as a fixed point

**The discount and the expectation applied by both steps once $\mathbb{T}$ is split.**

$$v_t = \max_c \left\{ u(c) + \beta \, \mathbb{E}_t \, v_{t+1}(\cdot) \right\} \qquad \text{read as} \qquad v_t = \max_c \left\{ u(c) + \beta^2 \, \mathbb{E}_{y'} \mathbb{E}_{y''} \, v_{t+1}(\cdot) \right\}.$$

Since the Bellman equation writes $\beta$ and $\mathbb{E}_t$ once each, an implementation that splits $\mathbb{T}$ into the choice and the expectation, as slide 44 does, must decide which step applies $\beta$ and which applies $\mathbb{E}_t$. An implementation that applies $\beta$ and $\mathbb{E}_t$ in each of the two steps makes the household discount at $\beta^2$ and draw income twice.

**Dated value functions read as one stationary function.**

$$v_t = \mathbb{T} \, v_{t+1}, \;\; v_{t+1} \text{ an input} \qquad \text{read as} \qquad v = \mathbb{T} \, v, \;\; \text{a fixed point.}$$

A stationarity condition that the statement never makes replaces the backward recursion from a terminal value. Once the terminal value has disappeared from the problem, the last period's consumption is chosen as if another period followed.

Appendix

# The Bellman calculus in full

# Stage, perch, field, block and builder

**Stage.** A stage is one unit of a dynamic programming problem, with three information sets: arrival, before the shock is drawn; decision, after it, where the choice is made; continuation, after the choice. One period of the buffer-stock problem is one stage. A period may list several stages, joined by connectors of the kind that slide 47 defines and slide 49 writes between periods.

**Perch.** A perch is one of the three information sets of a stage.

**Field.** A field is a state or a value declared at a perch.

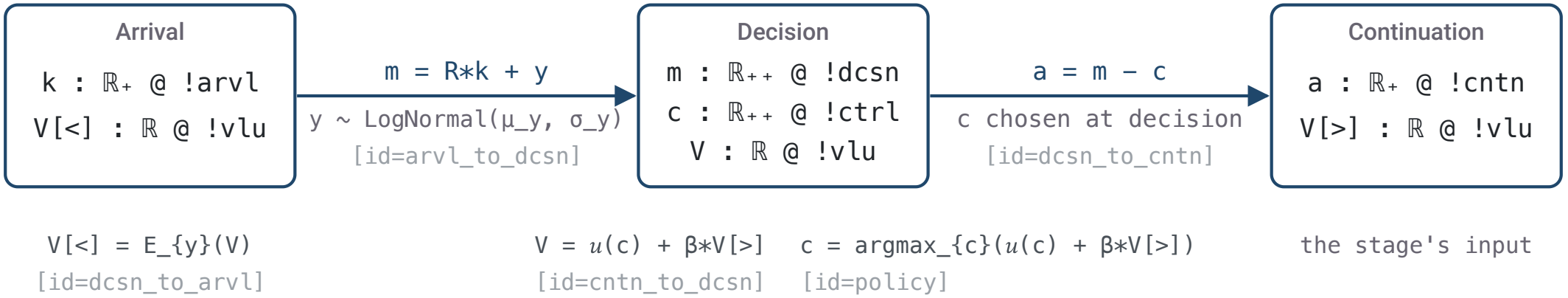
**Block.** The equations between declared inputs and outputs form a block.

**Builder.** Under Y (upsilon), a block denotes one operator, a builder.

# The stage file of the buffer-stock problem

The figure shows the stage file, the written form of slide 19, in the language Bellman-SYM.

**Written form, the typed stage file**




Three kinds of mark appear in the figure: `!arvl`, `!dcsn` and `!cntn` for the perches, `!ctrl` and `!vlu` for a control and a value, and `[id=…]` for the name of a block. The line `y ~ LogNormal(μ_y, σ_y)` declares the law of the shock. The language's reference on shock distributions defines `LogNormal(μ_y, σ_y)` as the law under which $\log y \sim N(\mu_y, \sigma_y^2)$, which fixes for every file the convention that slide 9 left to a subfield. `V[>]` at the continuation perch is the stage's input, supplied from outside the stage.

Bellman-SYM's documentation example (bellman-ddsl docs, Stages §4) in the letters k, m, a of the calculus, the perch marks written inside the type declarations, as The Bellman-SYM language §1.1 writes them · LogNormal: the reference on shock distributions.

# The preamble and one block of the stage file

The stage file of slide 42 opens with these lines, of which the figure shows only the law of the shock. The last three lines are one block, whose header lists its inputs and then its outputs.

```
β : (0,1)
ρ : (1, inf)
R : ℝ++
μ_y : ℝ
σ_y : ℝ+
y : ℝ++
y ~ LogNormal(μ_y, σ_y)

u(c : ℝ++) := c^(1-ρ)/(1-ρ)

[id=dcsn_to_arvl, (V) : !dcsn → (V[<] : ℝ @ !vlu) : !arvl] {
    V[<] = E_{y}(V)              :: [id=value]
}
```

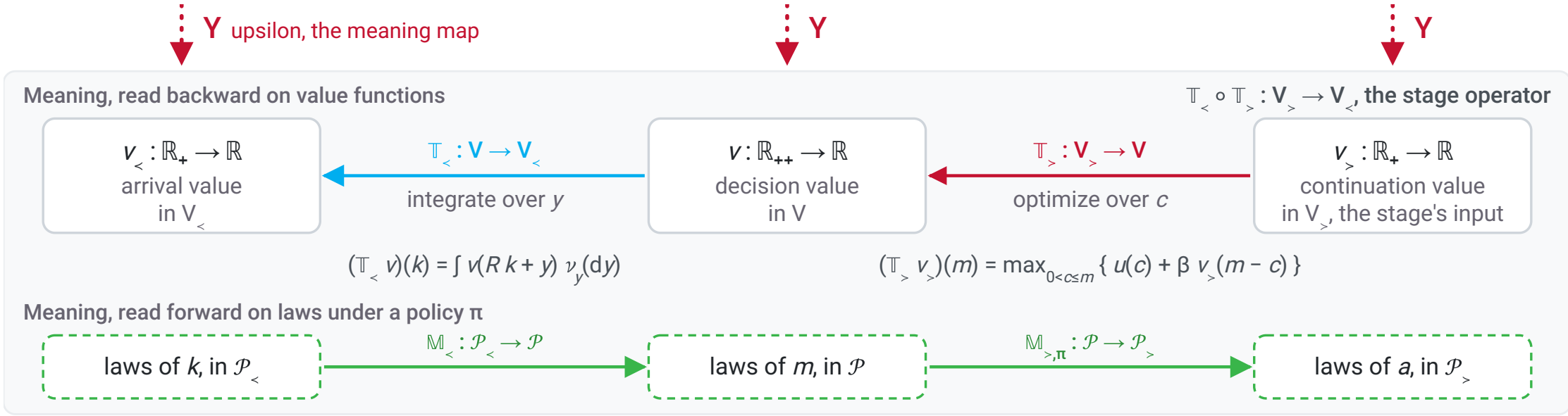
The file declares each parameter with its set. The lowercase ρ of the utility function is the curvature of utility, not P (rho), the map of slide 16 that attaches a numerical method.

A calibration file beside the stage file supplies the values of the parameters.

The declarations `c : ℝ++` and `a : ℝ+` together with the line `a = m - c` impose the bound $0 < c \leq m$ on the choice, over which the binder `argmax_{c}` , written without a set, then ranges.

# What the stage file denotes, backward on value functions and forward on laws

Under Y (upsilon), each value field `v` of the file denotes a function $v$ on the state space of its perch, an element of the set V of such functions, subscripted by the perch. The figure shows the three value functions with the two operators between them, and the laws on the three state spaces, elements of the sets $\mathcal{P}_<$, $\mathcal{P}$ and $\mathcal{P}_>$ of probability measures, which two forward operators map from one space to the next under the policy π, the rule $m \mapsto c(m)$ that the block `policy` defines.




The arrival block, where $y$ is drawn before the decision, fixes the date at which the shock enters the law of $m$. The file fixes the household's own shock only. An aggregate policy shock of the kind on slide 10 is no part of this stage.

The operators follow the Bellman calculus, Illustrative example §2 and Fields, functions and types §4 (bellman-ddsl/docs). The calculus writes $\mathrm{M}_\mathrm{p}$ for the probability measures on the state space $\mathrm{X}_\mathrm{p}$ (Fields, functions and types §4), whereas the deck writes $\mathcal{P}$ so that the letter M names only the forward operators. The dating of the shock: Bellman-SYM, Perch tagging rules §3.

# How the stage file refuses or makes visible the two readings of the expectation

Each reading of slide 37 is set beside the construct that refuses it or the line that shows it.

**The elaborator refuses a shock read beside its own expectation, whereas a shock replaced by its mean stands as a visible line.** Y (upsilon) interprets `y ~ LogNormal(μ_y, σ_y)`, `m = R*k + y` and `V[<] = E_{y}(V)` as

$$(\mathbb{T}_{\prec} v)(k) = \int v(Rk + y)\, \nu_y(\mathrm{d}y), \qquad y \text{ bound by the block, } \nu_y \text{ the declared law.}$$

Since an arrival value may depend only on what is known at arrival, where $y$ is not yet drawn, the line `V[<] = E_{y}(V) + y` cannot define one. The elaborator, the program that constructs the typed object from a file or refuses the file, refuses that line under the rule `shock-reintroduced`, since a shock that an expectation has integrated cannot appear again beside that expectation in a backward function. The other version of this reading, $y'$ replaced by its mean, cannot be written in this block either, since the elaborator refuses an expectation inside a transition under the rule `transition-without-expectation`. The mean enters only as a different transition line, $m = Rk + \exp(\mu_y + \sigma_y^2/2)$, which elaborates, that is, which the elaborator accepts, and stands visible.

---

**Since no block holds both the draw and the choice, the expectation cannot move through the maximization.** Next period's income is drawn in the next stage, after this stage's choice, which therefore cannot respond to it. The choice block `policy` lists `V[>]`, a function of savings, and no shock, which leaves $\mathbb{E}\max$ no block to be written in. A model in which a choice follows a draw declares the draw before the choice, an order the file then shows.

Bellman-SYM, Field equations §5, Perch tagging rules §3; the reference on refusals, rules shock-reintroduced (the line refused on 13 October 2026 with *y* listed among the block's inputs, whereas with *y* unlisted the rule signature-complete refuses it first) and transition-without-expectation (the mean inside the transition); the mean as a parameter in the transition elaborated the same day; Periods §3.1; bellman-ddsl wiki, Y §1 (the elaborator).

# How the Bellman calculus refuses a number for a function and a function on the wrong space

Each reading of slide 38 is set beside the construct that refuses it.

**A field is a function on its declared space, never a number.** Given the declaration `m : ℝ++ @ !dcsn` of a set and an information set, Y (upsilon) interprets `V = u(c) + β*V[>]` as

$$v(m) = u\big(c(m)\big) + \beta\, v_{\succ}\big(m - c(m)\big) \qquad \text{for every } m \in \mathbb{R}_{++},\ \ v_{\succ} \text{ on savings.}$$

A realized $\hat{m}_t$, having no form in the file, never enters the backward operators of slide 44, those on value functions. The laws of $k$ and $m$ enter only the forward operators, those on probability measures, which leaves the solver a single input, $v_{\succ} \in \mathsf{V}_{\succ}$, a function on the space the file declares.

---

**The elaborator refuses a continuation value listed at the decision perch.** Since `V[>] : ℝ @ !vlu` is listed at the continuation perch, whose state is `a : ℝ+` , under Y it denotes $v_{\succ} : \mathbb{R}_{+} \to \mathbb{R}$, a function of savings. A file that lists `V[>]` in a block header's list marked `!dcsn` , one of the two lists slide 43 shows, is refused under the rule `field-perch` , which requires that every listed field have one perch and that its mark, its side of the header and its role agree. A continuation value on resources therefore cannot be written.

Bellman-SYM, Stages §§2–4 and Appendix C (a field denotes a function of the state of its perch); the reference on refusals, rule field-perch, with its example of `V[>]` listed on a side marked `!dcsn` ; Bellman calculus, Introduction §§1–2 and Illustrative example §1 (bellman-ddsl/docs).

# Composition, and the discount once

The readings of slide 39 are set beside the construct that admits one composition only and the lines that show a second discount.

**Builders, the operators the blocks denote, are typed and compose in one order only.** Y (upsilon) interprets the block `cntn_to_dcsn`, together with `policy`, as the choice operator, the block `dcsn_to_arvl` of slide 43 as the expectation operator, and the stage as their composition,

$$\mathbb{T}_{\succ} : \mathsf{V}_{\succ} \to \mathsf{V}, \qquad \mathbb{T}_{\prec} : \mathsf{V} \to \mathsf{V}_{\prec}, \qquad \mathbb{T}_{\prec} \circ \mathbb{T}_{\succ} : \mathsf{V}_{\succ} \to \mathsf{V}_{\prec},$$

the only composition the types admit within the stage.

---

**The discount and the expectation each appear once, with any second copy visible on its own line.** `β` appears in the objective of the choice step, $u(c) + \beta\, v_{\succ}$, and not in the expectation block `V[<] = E_{y}(V)`. Because a connector, the entry `{a: k}` of the trellis file (slide 49, where that file lists the model's periods), identifies the continuation state of one stage with the arrival state of the next and performs no calculation, a join cannot produce $\beta^2$ or a second $\mathbb{E}_y$. A file that also writes `β` in the expectation block elaborates, with the second `β` standing on its own line. The objective is written twice, in the value equation and in the block `policy`. The elaborator lets the two differ, since the quasi-hyperbolic household of Laibson (1997) chooses under $\beta\delta$ and values under $\delta$, which leaves the modeler, not the elaborator, to keep them in agreement.

# The stationary closure

The last reading of slide 39, beside the lines that show it.

**Since the continuation input comes from outside the stage, a stationary closure, the condition $v_{t+1} = v_t$, is visible.** The next period or the terminal boundary data supplies `V[>]`, which leaves $v_{t+1} = v_t$ a condition the modeler adds and the stage does not impose. The trellis file of slide 49 sets the horizon at fifty periods, with a stationary closure written in none of the files.

# The period file and the trellis file

The period file places the stage file of slide 42 in a period, which the trellis file repeats and joins. The two files and the boundary data, not the stage, supply `V[>]` .

```
# period.yml
!period
name: buffer_stock_period
stages:
  - buffer_stock
connectors: []
```

```
# trellis.yml
!trellis
name: buffer_stock_life_cycle
periods:
  - {template: period.yml, repeat: 50}
inter_connectors:
  - {a: k}
```

The period lists one stage and no within-period connector. The trellis, the sequence of a model's periods counted backward from the terminal one, repeats the period fifty times. The inter-period connector `{a: k}` states that the savings `a` that leave one period are the assets `k` that enter the next, with no calculation. Since a stage file has no time index and is listed by name, one file serves every period that lists it.

The terminal period's `V[>]` is boundary data that neither file states and that P (rho) receives when it constructs the numerical problem. A bequest motive is a stage of the terminal period, as the durables model of the language's documentation writes it, which the comparator of slide 31 therefore reads. Only the continuation value after the last decision, zero for that model, stays outside the files, which makes two trellises that differ only in that constant one object to the comparator. A stationary closure has no entry in either file, since the horizon is the number of periods the trellis lists.

# The Bellman calculus and the Bellman operator

The paper's operator and the stage operator compose the same two parts, the expectation $\mathbb{T}_{\prec}$ and the choice $\mathbb{T}_{\succ}$:

| | | |
|---|---|---|
| the paper: | $\mathbb{T} = \mathbb{T}_{\succ} \circ \mathbb{T}_{\prec} : \mathsf{V} \to \mathsf{V},$ | the expectation, then the choice, across the period boundary; |
| the stage operator: | $\mathbb{T}_{\prec} \circ \mathbb{T}_{\succ} : \mathsf{V}_{\succ} \to \mathsf{V}_{\prec},$ | the choice, then the expectation, within the stage; |
| the identification: | $v_{\prec}$ of period $t+1$ is $v_{\succ}$ of period $t$, | since $k_{t+1} = a_t$. |

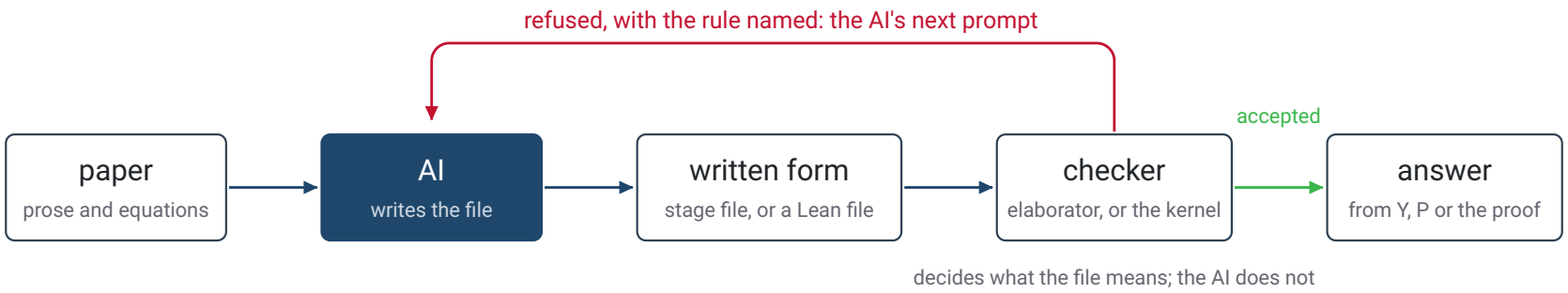
The two compositions differ only in where the period boundary falls. Y (upsilon) interprets the typed file as the stage composition $\mathbb{T}_{\prec} \circ \mathbb{T}_{\succ}$, to each part of which P (rho) attaches a numerical method. The continuation input comes from outside the stage, from the next period or a terminal boundary.

The elaborator refuses the readings that the file's types and blocks do not admit. The others, the mean of the shock in the transition and the second discount and stationary closure of slide 47, stand visible as lines the modeler must write, as explicit binding leaves them in a proof assistant such as Lean. The file proves nothing about the analysis. That the states suffice, that the integrals are finite, that a maximizer is measurable and that $\mathbb{T}$ has a unique fixed point remain theorems, stated and proved by the economist.

Appendix

# How an AI works with a semantic ontology

# The AI writes against the ontology




**Autoformalization.** The AI translates the paper into the written form, and the checker accepts or refuses the result (Wu et al. 2022; AlphaProof, Hubert et al. 2025, which learned by reinforcement with Lean's kernel as the verifier).

**Refuse and repair.** The checker's refusal, with its rule named, is the AI's next prompt, as a proof assistant's error message is in Baldur (First et al. 2023). The elaborator's rules `shock-reintroduced` and `field-perch` are messages of this kind.

**Constrained generation.** The AI's output is restricted at decoding time to the grammar of the written form, so that only its meaning remains to be checked (Poesia et al. 2022; Geng et al. 2023).

**Compute, then answer.** From the accepted file the ontology's programs compute, Y the object and P the numbers, or the kernel the proof, and the AI's answer to the user is built from that output rather than from its own guess.

**Round trip.** A second program writes the accepted object back into prose, and a judge compares that prose with the paper (Li et al. 2024; Allamanis et al. 2024). Matsya's architect mode runs this loop with the elaborator and two judges.

Wu, Jiang, Li, Rabe, Staats, Jamnik and Szegedy (2022), "Autoformalization with large language models", *NeurIPS 2022*, 32353–32368 · Hubert, Mehta, Sartran et al. (2025), *Nature* 651, 607–613 · First, Rabe, Ringer and Brun (2023), "Baldur: whole-proof generation and repair with large language models", *ESEC/FSE 2023*, 1229–1241 · Poesia et al. (2022), "Synchromesh", *ICLR 2022*; Geng, Josifoski, Peyrard and West (2023), "Grammar-constrained decoding for structured NLP tasks without finetuning", *EMNLP 2023*, 10932–10952 · Li, Wu, Li, Wei, Yang, Zhang and Ma (2024), "Autoformalize mathematical statements by symbolic equivalence and semantic consistency", *NeurIPS 2024*, 53598–53625; Allamanis, Panthaplackel and Yin (2024), "Unsupervised evaluation of code LLMs with round-trip correctness", *ICML 2024*, PMLR 235, 1050–1066 · Matsya's architect mode: bellman-ddsl, terminology, 9 October 2026.

# The AI reads from the ontology

**Retrieval over typed objects.** The AI answers from terms, definitions and relations retrieved from the ontology, each with a stated meaning, rather than from passages of text (Pan et al. 2024; Edge et al. 2024). Matsya's retrieval over the project's typed documents is this.

**Training on generated pairs.** The ontology generates pairs of a file and its prose, or the comparator labels two files as one object or as two, and the AI is trained on the pairs. AlphaProof's roughly one million natural-language problems, auto-formalized into about 80 million formal statements, are the largest instance, and the project's comparator module, `meaning_map_training` , is named for this use.

**What separates the two groups.** In the five uses of the previous slide the checker, not the AI, decides what the file means, so the AI's reading can be inspected. In these two the AI's reading improves but stays statistical, which is why they do not replace the checker.

Pan, Luo, Wang, Chen, Wang and Wu (2024), "Unifying large language models and knowledge graphs: a roadmap", *IEEE Transactions on Knowledge and Data Engineering* 36(7), 3580–3599 · Edge et al. (2024), "From local to global: a graph RAG approach to query-focused summarization", arXiv:2404.16130 · Hubert, Mehta, Sartran et al. (2025), *Nature* 651, 607–613, on the auto-formalized training problems · `meaning_map_training.compare` : bellman-ddsl.